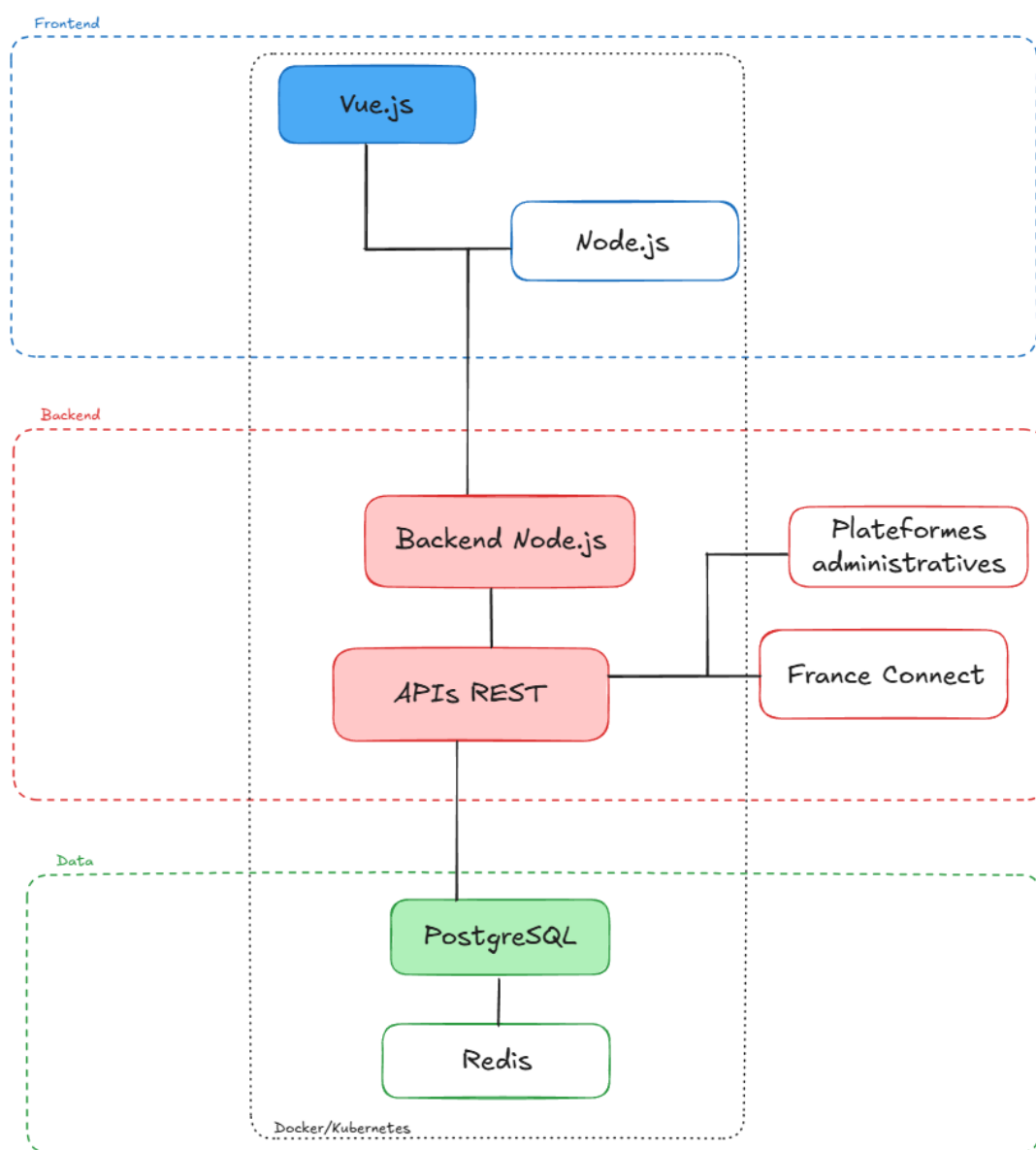


Jour 3/4 - Conception technique

Architecture système :

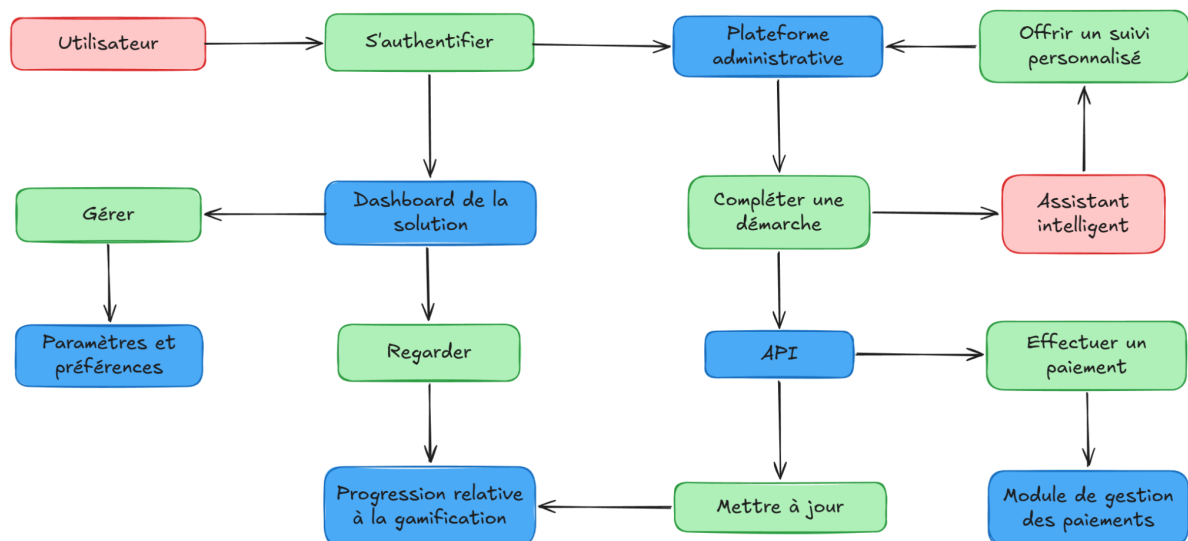
Au vu du niveau d'intérêt porté sur la sécurité, la rapidité et la fonctionnalité, une approche backend est indispensable.



Composants principaux :

- Authentification : Gérée exclusivement avec France Connect
- Profils : Liés aux profils France Connect, contiennent les données relatives à la gamification (points, badges, quêtes en cours/terminées, etc.). Un site dashboard sera créé pour permettre aux utilisateurs de gérer ces informations
- Assistant intelligent : Algorithme de machine-learning en premier lieu, plus tard changé pour un LLM quand et si plus de budget vient à cet effet
- Paiement : Informations toujours insuffisantes pour faire une description du module

Flux de données :



Stratégie :

Le design pattern à adopter sera un pattern MVC, celui-ci étant facilement maintenable, solide et généralement fortement compatible avec un système d'API REST.

Stack technique :

Front-end :

- [Vue.js](#) : Étant plus léger et performant que React, [Vue.js](#) est préféré dans le contexte d'une application avec une lourde emphase sur la rapidité de la plateforme.
- [i18next](#) : Une librairie facilitant l'ajout de plusieurs langues sur la plateforme. Même si l'ajout de langues supplémentaires n'est pas une priorité, le fonctionnement de cette librairie la rend intéressante à implémenter tôt, en sachant que des langues en plus du français peuvent être ajoutées plus tard.
- [Tailwind CSS](#) : Une librairie permettant de ne pas partir de zéro sur les feuilles de style de l'intégration. L'usage de Tailwind permettra de gagner du temps sur le développement tout en gardant en vue les contraintes du respect RGAA et de modernisation des vues.

Back-end :

- [Express](#) : Un framework efficace et standard de l'industrie, qui fonctionne bien avec PostgreSQL. L'usage du langage TypeScript plutôt que JavaScript est envisageable, pour des raisons de sécurité et de maintenabilité.

Data :

- [PostgreSQL](#) : Comme indiqué dans le cahier des charges. PostgreSQL est robuste et rapide, notamment pour les grandes quantités de données, ce à quoi nous sommes confrontés.
- [Redis](#) : Comme indiqué dans le cahier des charges. Redis permet de mettre des données en cache, augmentant encore plus les performances de notre base de données.

Conteneurisation :

- [Docker](#) : Comme indiqué dans le cahier des charges. Docker, côté développement, permet d'avoir un environnement partagé entre les développeurs, et surtout avec l'environnement de production. Il empêchera d'avoir des comportements inattendus en production qui n'auraient pas été détectés pendant le processus de développement.
- [Kubernetes](#) : Comme indiqué dans le cahier des charges. Kubernetes offre des outils parfaitement adaptés à notre architecture microservices, et permettant aussi de gérer la charge de l'application, ce qui sera très utile au fur et à mesure que le nombre d'utilisateurs augmente.

Modèle de données :

Voici un modèle de données simplifié, l'étendue des données utilisées exacte étant inconnue :

